A Verified Coin: Type Preservation for the Two-Move Hilbert System

Working paper v0.1, Project secret3, August 11, 2026

Abstract

The project's proof module carries a propositions-as-types coin for the two-move calculus: a Hilbert system whose axioms are the types of K and S, whose only rule is modus ponens, and whose derivations index the very terms that prove them. This paper adds the coin's dynamics. The theorem derivPreserved establishes subject reduction: if a term proves a proposition and takes one computation step, the result proves the same proposition; the corollary derivSteps extends this to whole runs, so that a proof remains a proof at every state of its execution. The proof is by inversion on rigid application spines, the module's house method, with injectivity of the arrow closing the type equations; all four step cases, the two redexes and the two congruences, are compiled, and the module remains total. A second contribution is an engineering finding about quantities: in a quantitative type theory, preservation cannot be proved over step relations with erased term indices, because the reconstructed derivations must materialize those terms; the paper reports the exact relevance refactor this forced, through the step relation, its closure, and the congruence layer, as a reusable lesson. The normalization half of the program, every typed term terminates, is stated as the sequel and not claimed here.

1. Introduction

The correspondence between proofs and programs has a static half and a dynamic half. The static half, that every derivation is a term and every term of the right type a derivation, is the coin the project minted in part I of its module and separated along its classical border in the Peirce paper [1]. The dynamic half is the subject of this paper: proofs are programs that can be run, and running them is harmless. Formally, subject reduction: typing is preserved by reduction. The result is classical for combinatory logic with simple types [2, 3]; the contribution here is its compilation for this project's particular coin, in a total module, with the proof architecture and the quantity lesson reported so that the book this program accompanies can point at a named theorem when it says, in its chapter on the laws of thought, that a proof of A implies A, set running on an input, is a proof at every step of its journey.

2. The coin, recalled

```
data Ty = TV Nat | Arr Ty Ty

data Deriv : Ty -> Term -> Type where
  AxK : {a, b : Ty} -> Deriv (Arr a (Arr b a)) CK
  AxS : {a, b, c : Ty} ->
        Deriv (Arr (Arr a (Arr b c)) (Arr (Arr a b) (Arr a c))) CS
  MP  : {a, b : Ty} -> {f, x : Term} ->
        Deriv (Arr a b) f -> Deriv a x -> Deriv b (App f x)
```

The positive inventory on this coin, iTyped, bTyped, wTyped, tTyped, and the negative border, peirceNoProof and dneNoProof, are in parts I, J, and L of the module and the companion paper [1].

3. The theorem and its architecture

```
derivPreserved : {c : Ty} -> Deriv c t -> Step t u -> Deriv c u
derivSteps     : {c : Ty} -> Deriv c t -> Steps t u -> Deriv c u
```

The proof of derivPreserved is one case analysis on the step, and each case is closed by inversion on a rigid spine, the same method the project's arithmetic papers used for their value ledger [4]. Three inversion lemmas do the work. An application is typed only by modus ponens, so derivAppInv extracts, from a derivation of App f x, the premise type and the two subderivations, packaged as a dependent pair. The two atoms carry only their axiom types, so kInvTy and sInvTy extract, from any derivation about K or S, the equation identifying its type with the axiom scheme. Injectivity of the arrow, two one-line lemmas, then turns those equations into the component identities the case needs.

The K case reads off the equations: inverting twice up the spine of K x y and once into the K derivation yields that the overall type equals the type of the kept argument x, and the derivation of x, transported along that identity, is the result. The S case is the same music with more voices: three inversions up the spine of S x y z, one into the S derivation, five arrow-injectivity components, and the result is assembled as MP (MP x′ z′) (MP y′ z′), each primed derivation the original transported along its component identity, the whole transported to the goal type. The two congruence cases invert once and rebuild: the stepped side is replaced by the recursive call, the spectator rides along. The corollary derivSteps is a two-line fold. Nothing in any case appeals to normalization, confluence, or any global property of reduction; preservation is local, and the proof is local with it.

4. The quantity lesson

The proof forced a refactor worth reporting, because it will recur for anyone doing this in a quantitative type theory. The module's step relation was originally declared with erased term indices, the natural default, since steps are usually only observed. Preservation, however, constructs: the S case must build MP nodes whose term indices are the very subterms the step rearranged, and in Idris2 those indices are relevant fields of the derivation. An erased index cannot be materialized, and the elaborator says so. The fix is principled rather than local: the step relation, its reflexive-transitive closure, and the congruence and transitivity helpers all acquire relevant term indices, after which every construction site in the module solves its terms by unification from relevant scope, and the whole 1,679-line module recompiles without further change. The lesson, stated once: a step relation you only watch may be erased; a step relation you rebuild typed terms from must carry its terms. The refactor is itself evidence for the module's design discipline, since fourteen prior parts survived a change to their foundation untouched.

5. What the theorem buys

Take the coin's first citizen, iTyped, the derivation that A implies A whose proof term is S K K. Apply it, by modus ponens, to any derivation d of A with term x: the result types App iC x at A. The module has long known that App iC x steps to x, the theorem skIdentity. What derivSteps adds is the reading the book wanted: along that run, every intermediate state, S K K applied to x, then K x (K x), then x, is itself a derivation of A, the proposition riding the computation like a passenger who never leaves the vehicle. Proofs are not merely programs; they are programs whose specification travels with them, step by step, and the checker has now signed that sentence.

6. Scope and the sequel

Three boundaries. The system is the K, S Hilbert coin over pure implication; richer signatures are future work. Progress, the claim that well-typed non-normal terms can always step, is true here and

not compiled, being unneeded for the book's uses. And the program's second half, normalization, every typed term reaches a normal form, is the L-grade sequel by Tait's method [5], registered on the research agenda as the remainder of line L1; this paper delivers the preservation half and defers that one explicitly.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 1,679 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification, and the entry point prints the theorem inventory. Part 0 is reproduced below; parts I, J, and L are in the companion paper [1].

Appendix B: The preservation section (module part 0)

```
--------------------------------------------------------------------
-- Tier 2, part 0: the coin preserved (L1, first half)
--------------------------------------------------------------------

arrInj1 : Arr a b = Arr c d -> a = c
arrInj1 Refl = Refl

arrInj2 : Arr a b = Arr c d -> b = d
arrInj2 Refl = Refl

||| an application is typed only by modus ponens
derivAppInv : Deriv c (App f x) ->
  (b : Ty ** (Deriv (Arr b c) f, Deriv b x))
derivAppInv (MP {a = b} df dx) = (b ** (df, dx))

||| K carries only its axiom type
kInvTy : Deriv ty CK -> (p : Ty ** q : Ty ** ty = Arr p (Arr q p))
kInvTy (AxK {a} {b}) = (a ** b ** Refl)

||| S carries only its axiom type
sInvTy : Deriv ty CS ->
  (p : Ty ** q : Ty ** r : Ty **
   ty = Arr (Arr p (Arr q r)) (Arr (Arr p q) (Arr p r)))
sInvTy (AxS {a} {b} {c}) = (a ** b ** c ** Refl)

||| subject reduction: one step of computation preserves the
||| proposition a proof term proves
derivPreserved : {c : Ty} -> Deriv c t -> Step t u -> Deriv c u
derivPreserved d (RedK {a = p0} {b = q0}) =
  let (b ** (dKp, dq)) = derivAppInv d
      (b2 ** (dK, dp)) = derivAppInv dKp
      (p ** q ** eq) = kInvTy dK
      e1 = arrInj1 eq
      e3 = arrInj2 (arrInj2 eq)
  in replace {p = \z => Deriv z p0} (sym (trans e3 (sym e1))) dp
derivPreserved d (RedS {a = p0} {b = q0} {c = r0}) =
  let (b ** (dSpq, dr)) = derivAppInv d
      (b2 ** (dSp, dq)) = derivAppInv dSpq
      (b3 ** (dS, dp)) = derivAppInv dSp
      (p ** q ** r ** eq) = sInvTy dS
      eB3 = arrInj1 eq
```

```
      eRest = arrInj2 eq
      eB2 = arrInj1 eRest
      eBC = arrInj2 eRest
      eB = arrInj1 eBC
      eC = arrInj2 eBC
      dp' = replace {p = \z => Deriv z p0} eB3 dp
      dq' = replace {p = \z => Deriv z q0} eB2 dq
      dr' = replace {p = \z => Deriv z r0} eB dr
  in replace {p = \z => Deriv z (App (App p0 r0) (App q0 r0))} (sym eC)
       (MP (MP dp' dr') (MP dq' dr'))
derivPreserved d (CtxL {l'} step) =
  let (b ** (df, dx)) = derivAppInv d
  in MP {f = l'} (derivPreserved df step) dx
derivPreserved d (CtxR {r'} step) =
  let (b ** (df, dx)) = derivAppInv d
  in MP {x = r'} df (derivPreserved dx step)

||| and whole runs preserve it: a proof stays a proof at every state
derivSteps : {c : Ty} -> Deriv c t -> Steps t u -> Deriv c u
derivSteps d Done         = d
derivSteps d (Then s ss)  = derivSteps (derivPreserved d s) ss
```

References

1. Project secret3, ‘No Bird Sings Peirce: A Machine-Checked Separation of Classical from Constructive Implication’, working paper v0.1, August 11, 2026.
2. H. B. Curry and R. Feys, *Combinatory Logic*, Vol. I, North-Holland, 1958.
3. J. R. Hindley, *Basic Simple Type Theory*, Cambridge University Press, 1997.
4. Project secret3, ‘Tree Numbers and Transparent Arithmetic’, working papers, August 2026.
5. W. W. Tait, ‘Intensional interpretations of functionals of finite type I’, Journal of Symbolic Logic 32 (1967), 198-212.
6. R. Atkey, ‘Syntax and semantics of quantitative type theory’, *Proceedings of LICS 2018*, ACM, 2018.
7. E. Brady, ‘Idris 2: Quantitative Type Theory in Practice’, *Proceedings of ECOOP 2021*, LIPIcs 194, 2021.